

.NET / ASP.NET Core Interview Questions & Answers

Extension Methods

Extension methods allow adding new methods to existing classes without modifying the original class. They are static methods inside a static class and use the 'this' keyword.

Example:

```
public static class StringExtension
{
    public static bool IsLong(this string value)
    {
        return value.Length > 10;
    }
}
```

Usage:

```
string name = "InterviewPreparation";
bool result = name.IsLong();
```

Middleware

Middleware are software components in ASP.NET Core request pipeline. Each middleware handles HTTP requests and responses.

Common middleware:

- Authentication
- Authorization
- Logging
- Exception handling

Example:

```
app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Before Middleware");
    await next();
    await context.Response.WriteAsync("After Middleware");
});
```

API Gateway

API Gateway acts as a single entry point for multiple microservices.

Responsibilities:

- Routing
- Authentication
- Rate limiting
- Load balancing
- Logging

Popular API Gateways:

- Ocelot
- YARP
- Kong
- NGINX

Filters vs Middleware

Middleware:

- Works in entire HTTP request pipeline
- Executes before MVC
- Used for logging, auth, exception handling

Filters:

- Works only inside MVC/Web API pipeline
- Used for action-level processing
- Types: Authorization, Action, Exception filters

Interview Answer:

Middleware is global for all requests while filters are specific to MVC actions/controllers.

Sort Array and Append Zeros at End

Question:

Sort array and move all zeros to end.

Example:

Input: [0,5,2,0,1]

Output: [1,2,5,0,0]

Code:

```
int[] arr = {0,5,2,0,1};

var result = arr.Where(x => x != 0)
    .OrderBy(x => x)
    .Concat(arr.Where(x => x == 0))
    .ToArray();
```

Providing Security to Requests

Ways to secure requests:

- HTTPS
- JWT Authentication
- OAuth
- API Keys
- Input Validation
- SQL Injection Prevention
- CORS Policy
- Rate Limiting

JWT Example:

```
services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer();
```

Rate Limiting

Rate limiting restricts number of requests from clients.

Purpose:

- Prevent DDOS attacks
- Prevent abuse
- Improve performance

ASP.NET Core Example:

```
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("fixed", opt =>
    {
        opt.PermitLimit = 10;
        opt.Window = TimeSpan.FromMinutes(1);
    });
});
```

LINQ Optimization

LINQ optimization techniques:

- Use Any() instead of Count()
- Use FirstOrDefault() carefully
- Avoid multiple enumerations
- Use Select only required columns
- Use AsNoTracking() in Entity Framework

```
Bad:
if(users.Count() > 0)

Good:
if(users.Any())
```

Take Only 5 Elements from LINQ

```
Example:
var topFive = users.Take(5).ToList();

Take() retrieves only required records improving performance.
```

Middleware Use, Run and Map

```
Use():
- Passes request to next middleware.
- Most commonly used.
```

```
Run():
- Terminal middleware.
- Stops pipeline execution.
```

```
Map():
- Branches pipeline based on URL.
```

Example:

```
app.Use(async (context, next) =>
{
    await next();
});

app.Map("/admin", adminApp =>
{
    adminApp.Run(async context =>
    {
        await context.Response.WriteAsync("Admin Page");
    });
});

app.Run(async context =>
{
    await context.Response.WriteAsync("Main Request");
});
```

Common Interview Questions

1. Difference between IEnumerable and IQueryable?

Answer:
IEnumerable executes query in memory.
IQueryable executes query in database.

2. What is Dependency Injection?

Answer:
Technique for achieving loose coupling by injecting dependencies.

3. What is async/await?

Answer:
Used for asynchronous programming without blocking threads.

4. What is JWT?

Answer:
JSON Web Token used for authentication and authorization.

5. What is CORS?

Answer:
Cross-Origin Resource Sharing allows frontend/backend communication from different domains.